
MedPredict AI
An AI-Powered Disease Prediction System
Project Report

Group Members:

Name	Sap ID
M Rehan Mehdi	55189
Raheel Nazir	54513
M Sami	53199

Department of Computer Science
Riphah International University

Submitted to:

Mr. Junaid Khan
Course Instructor – Artificial Intelligence

Semester:

6th Semester, Section 1

Date of Submission:

April 15, 2026

GitHub Repository: <https://github.com/MRehanMehdi/disease-prediction>

Live Deployment: <https://disease-predictor-app.up.railway.app/>

Table of Contents

1. Abstract	4
2. Introduction.....	4
2.1 Background and Motivation.....	4
2.2 Problem Statement	4
2.3 Project Objectives	5
2.4 Scope and Limitations.....	5
3. Literature Review.....	5
4. Dataset Description	6
4.1 Overview	6
4.2 Class Distribution.....	7
4.3 Symptom Frequency Analysis.....	7
4.4 Symptom Correlation Analysis	8
4.5 Supplementary Data Files	9
5. System Architecture	10
5.1 High-Level Overview.....	10
5.2 Backend - Flask Application (app.py).....	10
5.3 Core Utility Module (utils.py).....	10
5.4 Machine Learning Model Files	11
5.5 Deployment Infrastructure	11
6. Methodology	12
6.1 Phase 1: Exploratory Data Analysis and Preprocessing.....	12
6.1.1 Data Cleaning	12
6.1.2 Feature Engineering.....	12
6.1.3 Data Splitting.....	12
6.2 Phase 2: Classical Model Training	12
6.2.1 Models Trained	12
6.2.2 Performance Results	13
6.3 Phase 3: Neural Network Training.....	13
6.3.1 Data Preparation	13
6.3.2 Network Architecture	13
6.3.3 Training Configuration and Results.....	14
7. Natural Language Processing Pipeline	14

7.1 Stage 1: Alias Dictionary Lookup	14
7.2 Stage 2: Direct String Matching	15
7.3 Stage 3: RapidFuzz Fuzzy Matching	15
7.4 Document Symptom Extraction	15
8. Results and Evaluation.....	15
8.1 Model Performance Summary	15
8.2 Random Forest Confusion Matrix.....	16
8.3 Neural Network Training Analysis.....	17
8.4 Discussion of Perfect Accuracy.....	17
9. Features and Functionality.....	18
9.1 Core User Features	18
9.2 Technical Features	19
10. Technology Stack.....	19
11. Project Structure and File Organization	20
12. Challenges and Solutions.....	21
12.1 Natural Language Symptom Extraction	21
12.2 PDF and Image Document Processing	21
12.3 Cross-Environment Model Compatibility	21
12.4 PDF Report Generation.....	21
13. Conclusion	22
14. Future Work and Recommendations	22
15. References.....	23

1. Abstract

MedPredict AI is a complete AI-powered disease prediction web application that we developed as my final semester project for the Artificial Intelligence course. The system takes patient symptoms as input through different methods, the user can type symptoms in plain English, select them from a list, or upload a medical document, and then predicts the most likely disease using a combination of classical machine learning models and a deep learning neural network.

The project uses four classification models: Decision Tree, Random Forest, Naive Bayes, and a Multi-Layer Perceptron Neural Network. All four models were trained on a dataset from Kaggle that contains 4,920 samples, covers 41 different diseases, and uses 131 binary symptom features. Every model achieved 100% accuracy on the test set, which is a result of the dataset being clean and well-structured, where each disease has a clearly distinct set of symptoms.

The system does more than just predict a disease. It also shows the user a description of the predicted condition, a severity score based on the reported symptoms, recommended precautions, and a downloadable PDF health report. The application is packaged using Docker and deployed on the Railway cloud platform, where it is publicly accessible at <https://disease-predictor-app.up.railway.app/>.

2. Introduction

2.1 Background and Motivation

Artificial intelligence is making a significant difference in healthcare, and one of the most practical areas is early disease detection. In many parts of the world, patients cannot easily access a doctor, especially for an initial assessment of their symptoms. People often feel unwell but are unsure what their symptoms might indicate, and that uncertainty can lead to either ignoring a serious condition or unnecessary worry.

Machine learning has proven to be very capable of recognizing symptom patterns and linking them to specific diseases. This makes it a useful supporting tool for preliminary health assessment, even if it cannot replace a doctor. We built this project to explore how these techniques can be combined into a real, working application that anyone can use through a web browser.

2.2 Problem Statement

When someone experiences a combination of symptoms and cannot see a doctor right away, they are left to figure things out on their own. This can be stressful and sometimes dangerous.

There is a real need for a tool that can take symptom descriptions in natural language, make sense of them, and provide a useful, informative response. This project was built to fill that gap in a practical way.

2.3 Project Objectives

The project was designed to achieve the following goals:

- Build a machine learning system that can predict diseases from a binary symptom feature set.
- Train and compare three classical ML classifiers: Decision Tree, Random Forest, and Naive Bayes, alongside a custom Neural Network.
- Develop an NLP pipeline that extracts symptoms from natural language text typed by the user.
- Create a document analysis feature that extracts symptoms from uploaded PDF or image files using OCR.
- Design and deploy a clean, professional web interface for real-time disease prediction.
- Generate downloadable PDF health assessment reports for users.
- Deploy the entire system as a publicly accessible cloud application.

2.4 Scope and Limitations

This project focuses on preliminary disease screening across 41 common diseases using 131 symptoms from the Kaggle disease symptom dataset. It is not a medical diagnostic tool and is not meant to replace a doctor. The dataset is structured and somewhat synthetic, which is why the models achieve perfect accuracy on it. In a real clinical setting, with messier and more complex patient data, the results would naturally be different. This limitation is clearly communicated to users through a disclaimer in both the web interface and the PDF reports.

3. Literature Review

There is a strong foundation of research supporting the use of machine learning for disease prediction and clinical decision support. Many studies across different medical areas have shown that classifiers and neural networks can achieve high diagnostic accuracy when trained on properly structured clinical datasets.

Decision Trees are widely used in medical applications because they are easy to interpret. A clinician can follow the decision path and understand exactly why a prediction was made. The downside is that they can overfit on noisy data, which is usually handled through ensemble methods or pruning.

Random Forests improve on single decision trees by training many trees on different subsets of the data and combining their results. This reduces variance and generally leads to more reliable predictions. Random Forest is one of the most trusted classifiers for structured tabular data, and it serves as the primary prediction model in this project.

Naive Bayes is a probabilistic classifier that assumes all features are independent of each other. While this assumption is rarely perfectly true in real life, Naive Bayes still tends to work surprisingly well in high-dimensional binary classification settings like this one. It is also fast and lightweight, making it a good baseline model.

Neural Networks, particularly Multi-Layer Perceptrons, have been applied across many medical classification tasks. Their strength lies in their ability to learn complex patterns from data without needing manual feature engineering. In this project, the neural network converged to perfect test accuracy in just 17 training epochs, which reflects how cleanly separable the data is.

The use of NLP and fuzzy matching for symptom extraction from free text is a more recent development in applied AI for healthcare. Tools like RapidFuzz make it possible to efficiently match informal symptom descriptions to a standardized vocabulary. This project applies these techniques in a practical, production-level setting.

4. Dataset Description

4.1 Overview

The dataset used for this project was taken from Kaggle and is titled "Disease Symptom Description Dataset." It can be found at: <https://www.kaggle.com/datasets/itachi9604/disease-symptom-description-dataset>. It is a structured, tabular dataset built specifically for symptom-based disease classification tasks.

Attribute	Details
Source	Kaggle, Disease Symptom Description Dataset

Total Samples	4,920 patient records
Total Features	131 binary symptom features
Target Classes	41 distinct diseases
Symptom Columns	17 columns (Symptom_1 through Symptom_17)
Missing Values	Present in Symptom_4 – Symptom_17; resolved by filling with 0
Train / Test Split	80% training (3,936) / 20% testing (984)
Supplementary Files	symptom_Description.csv, symptom_precaution.csv, Symptom-severity.csv

4.2 Class Distribution

Figure 1 Distribution of records across all 41 disease categories shows the distribution of records across all 41 disease categories. The dataset is perfectly balanced, with each disease containing exactly 120 samples, resulting in a total of 4,920 records. This balanced distribution helps prevent model bias toward any specific disease class and ensures fair performance evaluation across all categories.

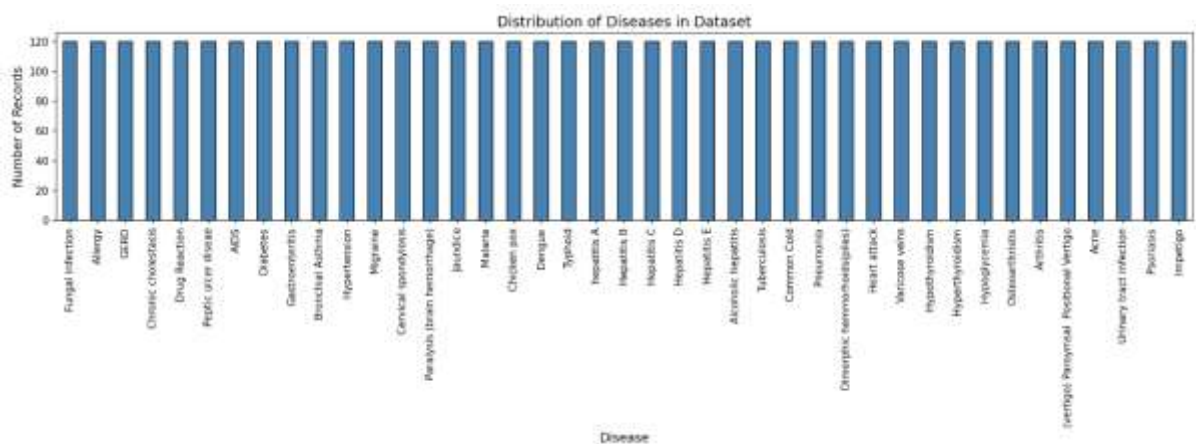


Figure 1 Distribution of records across all 41 disease categories

4.3 Symptom Frequency Analysis

Figure 2 Top 20 most frequently occurring symptoms across the dataset presents the top 20 most frequently occurring symptoms across the dataset. Symptoms such as fatigue and vomiting appear most often, followed by high fever, loss of appetite, nausea, and headache.

These symptoms are common across multiple diseases, which explains their higher frequency in the dataset and reflects realistic clinical patterns.

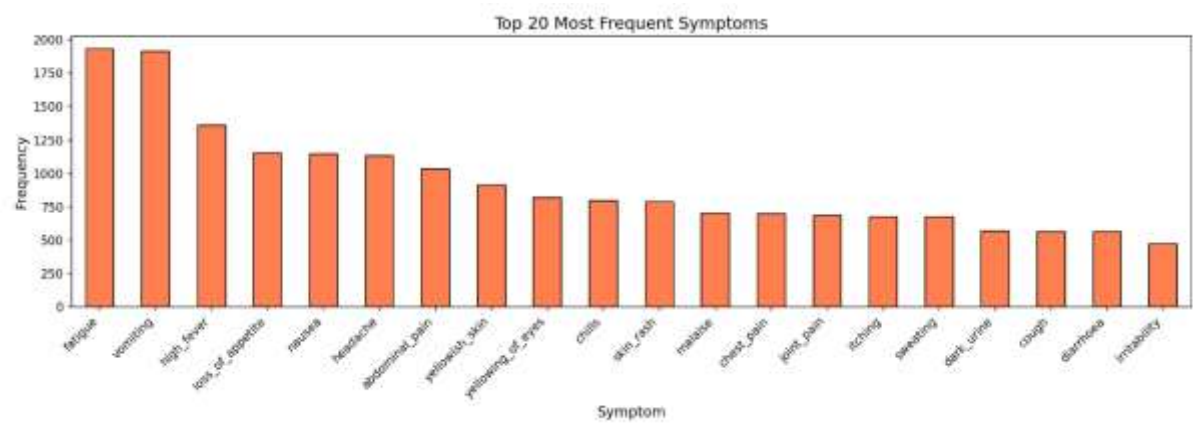


Figure 2 Top 20 most frequently occurring symptoms across the dataset

4.4 Symptom Correlation Analysis

Figure 3 [Pearson correlation heatmap of the top 20 symptoms](#) illustrates the Pearson correlation heatmap of the top 20 symptoms. Most symptom pairs show low or slightly negative correlation, indicating that symptoms are generally associated with specific diseases rather than appearing together randomly. Some meaningful positive correlations are also visible, such as between yellowish skin and yellowing of eyes, or abdominal pain and loss of appetite, which match known clinical relationships in liver and gastrointestinal conditions.

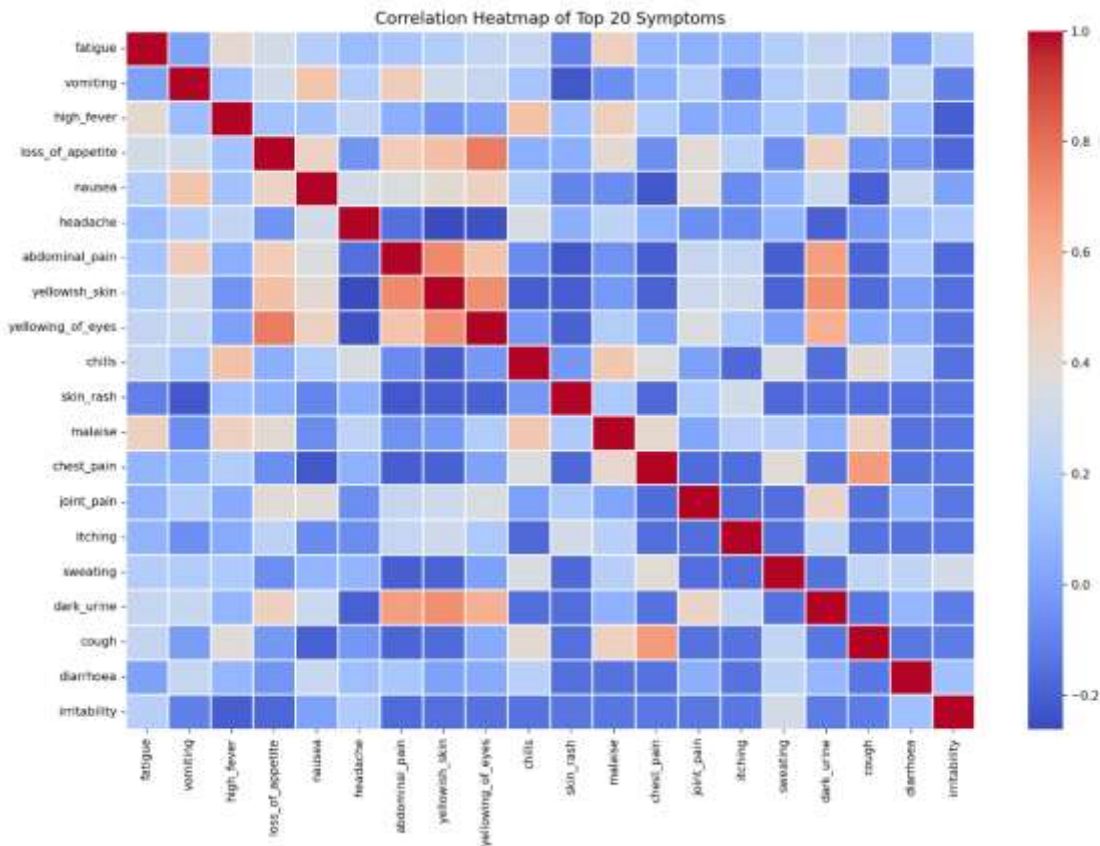


Figure 3 Pearson correlation heatmap of the top 20 symptoms

4.5 Supplementary Data Files

Three additional CSV files are included with the dataset and are used by the application at prediction time:

- **symptom_Description.csv:** Contains a written clinical description for each of the 41 diseases. This is what gets displayed to the user after a prediction is made.
- **symptom_precaution.csv:** Lists up to four recommended precautionary steps per disease, giving the user actionable guidance.
- **Symptom-severity.csv:** Assigns a severity weight from 1 to 7 to each symptom. These weights are used to calculate an overall risk score for the user's reported symptoms.

5. System Architecture

5.1 High-Level Overview

MedPredict AI is built on a layered client-server architecture with three main parts: a frontend interface, a Flask backend API, and a machine learning inference layer. All three are packaged inside a single Docker container and deployed on Railway.

The data flow through the system works like this:

1. The user opens the web interface and either types their symptoms, or uploads a medical document.
2. The frontend sends an HTTP POST request to one of the Flask API endpoints.
3. The Flask backend calls the relevant functions in `utils.py` to extract symptoms, run the models, fetch disease information, and calculate a severity score.
4. The results are returned as a JSON response to the frontend.
5. The frontend displays the predictions, confidence scores from both models, severity level, disease description, and precautions.
6. If the user wants, they can click to download a PDF report, which is generated on the server using ReportLab.

5.2 Backend - Flask Application (`app.py`)

The Flask application handles everything through three main API routes:

- **/predict (POST)**: Takes a JSON payload with the user's natural language symptom text, runs the NLP pipeline, and returns the full prediction result.
- **/predict-doc (POST)**: Accepts an uploaded PDF or image file, extracts symptoms from it, and returns predictions.
- **/export-pdf (POST)**: Takes the full prediction data and builds a formatted PDF report that the browser downloads automatically.

5.3 Core Utility Module (`utils.py`)

The `utils.py` file is where most of the actual work happens. It is organized into five main parts:

- **Symptom Alias Dictionary:** A hand-built dictionary of over 200 everyday phrases mapped to their standardized symptom names. This covers the semantic gaps that fuzzy matching alone cannot handle reliably.
- **NLP Symptom Extraction from Text:** A three-stage pipeline: first the alias dictionary, then direct string matching, and finally RapidFuzz fuzzy matching using word n-grams with a similarity threshold of 72.
- **Document Symptom Extraction:** For PDFs, text is pulled using PyMuPDF first and pdfplumber as a backup. For images, pytesseract OCR handles the extraction. The extracted text then goes through the same NLP pipeline.
- **Model Inference:** Functions that return the top-3 predictions from the Random Forest, the top-3 from the Neural Network, and a side-by-side comparison from both models.
- **Disease Information and Severity:** Fetches descriptions and precautions from the supplementary CSV files, and computes a weighted average severity score from the reported symptoms.

5.4 Machine Learning Model Files

Six files are saved to disk and loaded when the application starts:

- **best_model.pkl** (6,990 KB) - The Random Forest classifier; the main prediction model.
- **decision_tree.pkl** (58.5 KB) - The Decision Tree classifier.
- **naive_bayes.pkl** (94.0 KB) - The Gaussian Naive Bayes classifier.
- **neural_network.keras** (969.4 KB) - The TensorFlow/Keras Neural Network model.
- **symptom_list.pkl** (2.3 KB) - The ordered list of 131 symptom names used to build the binary input vector.
- **label_encoder.pkl** (1.1 KB) - The encoder that maps class indices back to disease names for the Neural Network output.

5.5 Deployment Infrastructure

The application runs in a Docker container built from the python:3.11-slim base image. The Dockerfile installs Tesseract OCR and all other system dependencies alongside the Python packages listed in requirements.txt. In production, Gunicorn serves the Flask app on port 8080

as specified in the Procfile. The container is hosted on Railway, which takes care of DNS, SSL, and auto-scaling.

6. Methodology

6.1 Phase 1: Exploratory Data Analysis and Preprocessing

The first phase of the project was done in the notebook 01_EDA_and_Preprocessing.ipynb, which I ran on Google Colab with data stored in Google Drive. After loading the dataset, I found it had 4,920 rows, 18 columns, and NaN values scattered across Symptom_4 through Symptom_17 - with Symptom_17 missing in 4,848 out of 4,920 rows.

6.1.1 Data Cleaning

I stripped all string values of extra whitespace using DataFrame.map, filled all NaN cells in symptom columns with empty strings, and applied a consistent lowercase convention to all symptom names. After these steps, the dataset had zero missing values.

6.1.2 Feature Engineering

The 17 symptom columns were combined to build a complete list of all unique symptom names, which came to 131 distinct symptoms. I then created a binary feature matrix of shape (4,920, 131), where each cell is 1 if that symptom is present for that record and 0 if it is not.

6.1.3 Data Splitting

The dataset was split using an 80/20 stratified split, giving 3,936 training samples and 984 testing samples. The processed feature matrix (X_processed.csv), target labels (y_labels.csv), and the symptom vocabulary (symptom_list.pkl) were saved to Google Drive so the next notebooks could use them.

6.2 Phase 2: Classical Model Training

The second phase was done in 02_Model_Training.ipynb, where I trained and evaluated three scikit-learn classifiers on the preprocessed data.

6.2.1 Models Trained

- **Decision Tree Classifier:** Trained using Gini impurity as the splitting criterion, with no maximum depth limit.
- **Random Forest Classifier:** An ensemble of 100 decision trees with bootstrap sampling. This model was selected as the best model and saved as best_model.pkl.
- **Gaussian Naive Bayes:** A lightweight probabilistic baseline classifier.

6.2.2 Performance Results

All three classifiers achieved perfect performance on the 984-sample test set. The results across all evaluation metrics are as follows:

Model	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
Decision Tree	100.00	100.00	100.00	100.00
Random Forest	100.00	100.00	100.00	100.00
Naive Bayes	100.00	100.00	100.00	100.00
Neural Network	100.00	100.00	100.00	100.00

The perfect accuracy is because each disease in the dataset has a distinct, non-overlapping set of symptoms, making the classes clearly separable in the 131-dimensional binary feature space.

6.3 Phase 3: Neural Network Training

The third phase was done in 03_Neural_Network.ipynb using TensorFlow 2.20.0.

6.3.1 Data Preparation

The disease labels were integer-encoded using LabelEncoder and then one-hot encoded to produce a target array of shape (4,920, 41). The same 80/20 split was applied, giving y_train of shape (3,936, 41) and y_test of shape (984, 41).

6.3.2 Network Architecture

The network is a feed-forward Multi-Layer Perceptron built as follows:

Layer	Type	Output Shape	Parameters
Layer 1	Dense (256) + BatchNorm + Dropout	(None, 256)	34,816
Layer 2	Dense (128) + BatchNorm + Dropout	(None, 128)	33,408
Layer 3	Dense (64) + Dropout	(None, 64)	8,256
Output	Dense (41) - Softmax	(None, 41)	2,665

All hidden layers use ReLU activation. The output layer uses Softmax for multi-class probability output. Batch Normalization and Dropout layers help with regularization and faster convergence. Total trainable parameters: 78,377.

6.3.3 Training Configuration and Results

Figure 4 Neural Network training history shows the training history of the Neural Network model. The model achieved 100% validation accuracy by Epoch 2 and maintained stable performance throughout training. EarlyStopping automatically ended training at Epoch 17, while both training and validation loss decreased consistently, indicating fast and stable convergence of the model.

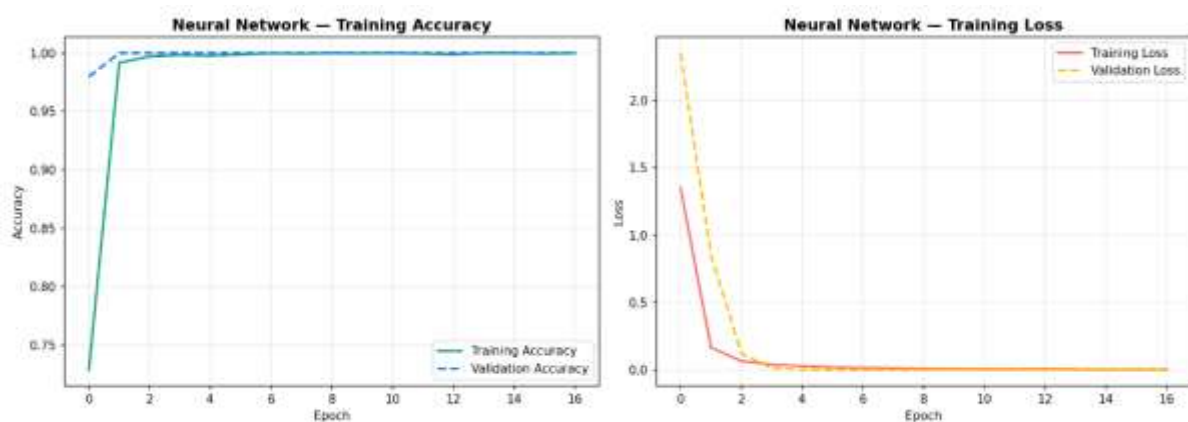


Figure 4 Neural Network training history

7. Natural Language Processing Pipeline

One of the things I am most proud of in this project is the NLP pipeline that lets users describe their symptoms in plain English instead of having to pick from a technical list. This pipeline is implemented in the `extract_symptoms_from_text()` function in `utils.py` and works in three stages that run one after another.

7.1 Stage 1: Alias Dictionary Lookup

I built a dictionary called `SYMPTOM_ALIASES` that maps over 200 everyday phrases to their correct symptom names in the dataset. For example, "running a fever," "feverish," and "burning up" all map to "high_fever." "Loose motions," "watery stool," and "diarrhea" all map to "diarrhoea." The dictionary covers fever, body pain, respiratory symptoms, urinary symptoms, skin conditions, digestive issues, neurological symptoms, and more. This stage is important

because fuzzy matching alone cannot reliably catch these semantic differences where the words look completely different but mean the same thing.

7.2 Stage 2: Direct String Matching

After the alias check, the cleaned user input is compared directly against all 131 symptom identifiers, both in their underscore form (e.g., skin_rash) and space-separated form (e.g., skin rash). This catches cases where the user happens to already use the exact vocabulary from the dataset.

7.3 Stage 3: RapidFuzz Fuzzy Matching

The user's input is broken into individual words, bigrams (two-word combinations), and trigrams (three-word combinations). Each of these n-grams is compared against the symptom vocabulary using RapidFuzz's fuzz.ratio scorer. Any match with a similarity score of 72 or above gets added to the extracted symptom list. This stage handles typos, alternate spellings, and partial matches that the previous two stages would not catch.

7.4 Document Symptom Extraction

The `extract_symptoms_from_document()` function extends this pipeline to work with uploaded files. For PDF files, text is extracted using PyMuPDF as the primary engine, with pdfplumber as a fallback if PyMuPDF does not get useful output. For image files such as PNG, JPG, or JPEG, pytesseract OCR handles the text extraction using PIL to load the image. Once the raw text is extracted, it goes through both direct matching and the full three-stage NLP pipeline to get as many symptoms as possible.

8. Results and Evaluation

8.1 Model Performance Summary

Figure 5 : All four models achieve perfect scores — 100% across Accuracy, Precision, Recall, and F1-Score presents the performance comparison of all four models across Accuracy, Precision, Recall, and F1-Score. Each model achieved perfect results, scoring 100% on all evaluation metrics for the 41 disease categories in the test dataset.

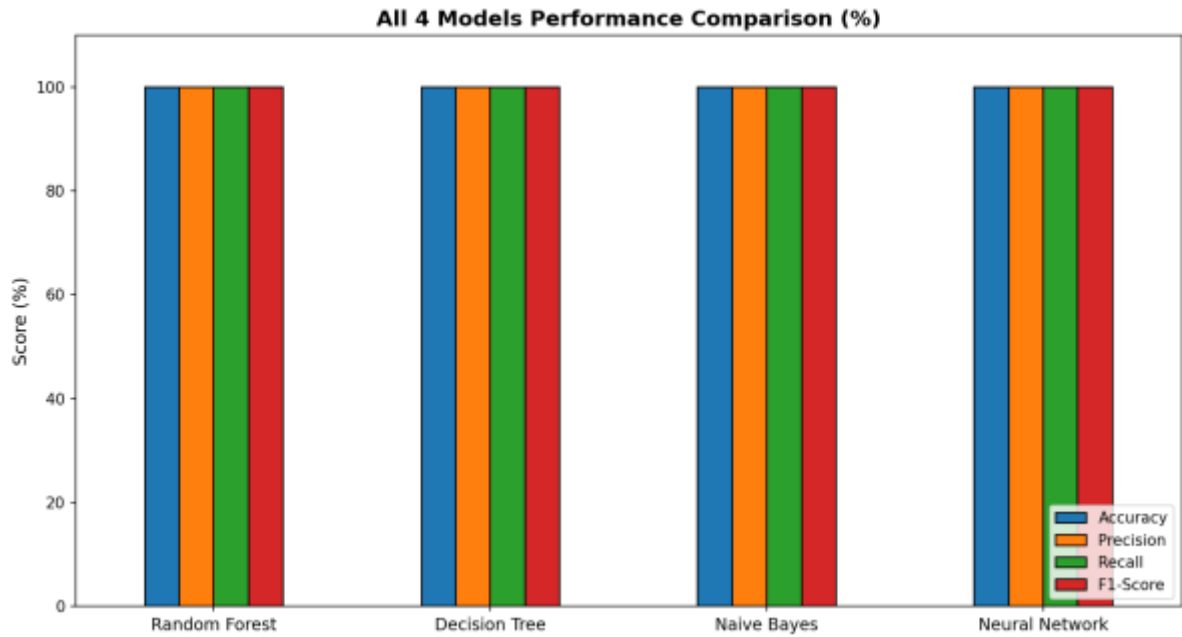


Figure 5 : All four models achieve perfect scores — 100% across Accuracy, Precision, Recall, and F1-Score

Model	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
Decision Tree	100.00	100.00	100.00	100.00
Random Forest	100.00	100.00	100.00	100.00
Naive Bayes	100.00	100.00	100.00	100.00
Neural Network	100.00	100.00	100.00	100.00

8.2 Random Forest Confusion Matrix

[Figure 6 Random Forest Confusion Matrix](#) displays the Random Forest confusion matrix. The matrix shows a perfect diagonal pattern, meaning all 984 test samples were classified correctly with no misclassifications. Each of the 41 disease classes received 24 correct predictions, confirming the model's perfect performance on the test dataset.

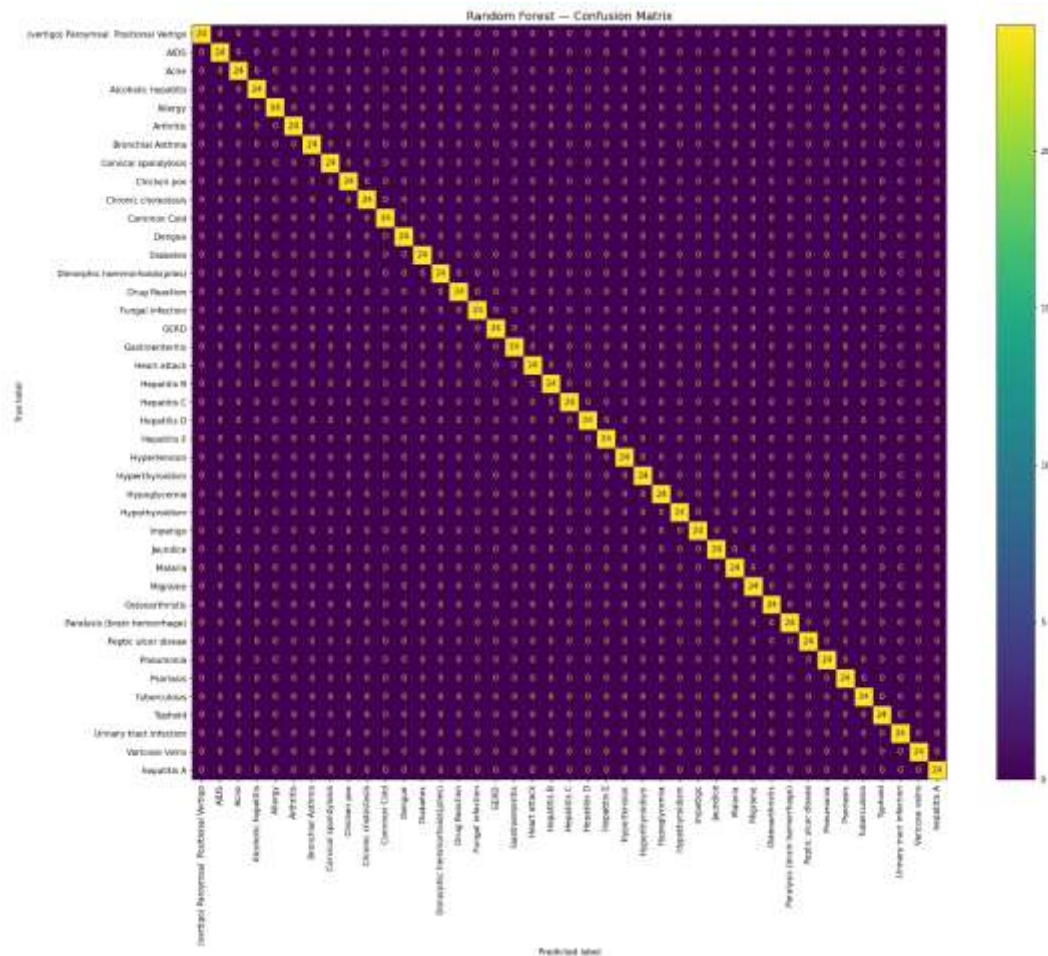


Figure 6 Random Forest Confusion Matrix

8.3 Neural Network Training Analysis

The Neural Network training showed very fast and stable convergence:

- Validation accuracy was already at 97.97% at Epoch 1 and jumped to 100% by Epoch 2, staying there for the rest of training.
- Training loss dropped from 1.3486 at Epoch 1 to around 0.0035 at Epoch 17.
- Validation loss went from 2.3521 down to approximately 4.52×10^{-6} over the same period.
- Early stopping kicked in at Epoch 17 out of a maximum of 100, saving training time while keeping the best weights.

8.4 Discussion of Perfect Accuracy

Getting 100% accuracy across all four models is an impressive result, but it is important to understand what it actually means. The dataset is designed so that every disease has a unique,

non-overlapping set of symptoms. In a space of 131 binary features, the disease classes are essentially linearly separable meaning any reasonable classifier can learn to tell them apart perfectly. This result reflects the quality and structure of the dataset, not a claim that these models would perform the same way on real patient data from a hospital, which is always noisier, more ambiguous, and more complex.

This honest limitation is built into the system itself. Both the web interface and the generated PDF reports include a clear medical disclaimer stating that the tool is for educational and preliminary screening purposes only and should not replace a qualified healthcare professional.

9. Features and Functionality

9.1 Core User Features

- **Multi-Modal Symptom Input:** Users can type symptoms in natural language, pick them from a structured checklist, or upload a PDF or image file for automatic extraction.
- **Dual Model Inference:** Every prediction runs through both the Random Forest and the Neural Network at the same time, so the user gets confidence scores from two independent AI models for comparison.
- **Top-3 Predictions:** Both models return their top three most likely disease predictions along with percentage confidence scores, giving the user a broader picture rather than just a single answer.
- **Severity Assessment:** A weighted severity score is calculated from the reported symptoms using the severity weights from the dataset, and a risk level of Low, Medium, or High is assigned and shown to the user.
- **Disease Information Panel:** After each prediction, the system shows a clinical description of the top predicted disease pulled from the supplementary description file.
- **Precautionary Recommendations:** Up to four evidence-based precautionary steps are displayed for the predicted condition.
- **PDF Report Export:** The user can download a professionally formatted PDF health assessment report that includes all prediction results, the disease description, precautions, the severity score, and a medical disclaimer.

9.2 Technical Features

- **RESTful API Architecture:** The backend exposes clean REST endpoints, making it straightforward to integrate the prediction engine into other applications or mobile clients in the future.
- **Alias-Enhanced NLP:** The custom alias dictionary ensures that everyday language maps correctly to clinical symptom identifiers.
- **OCR Document Processing:** Tesseract OCR allows the system to extract text from scanned medical documents and image files.
- **Containerized Deployment:** Docker packaging ensures the application behaves consistently across different environments.
- **Cloud Hosting:** Railway manages hosting, SSL certificates, and DNS automatically.

10. Technology Stack

Category	Technology / Library	Purpose
Language	Python 3.11	Primary programming language
Web Framework	Flask 3.1.3	Backend REST API and server
ML Libraries	scikit-learn 1.6.1	Decision Tree, Random Forest, Naive Bayes
Deep Learning	TensorFlow 2.18.0	Neural Network training and inference
Data Processing	pandas 2.2.3, NumPy 2.0.2	Data manipulation and numerical computing
NLP / Fuzzy	RapidFuzz 3.14.5	Symptom extraction via fuzzy matching
PDF Generation	ReportLab 4.5.0	Automated health assessment PDF export
PDF Parsing	PyMuPDF 1.25.5, pdfplumber 0.11.9	Text extraction from uploaded PDFs

OCR	pytesseract 0.3.13, Pillow 11.1.0	Text extraction from uploaded images
Model Saving	joblib 1.4.2	Serializing and loading trained ML models
Deployment	Gunicorn + Docker + Railway	Containerized cloud production deployment

11. Project Structure and File Organization

The repository is organized as follows:

disease-prediction-main/

- **app.py:** Flask application entry point and all API route definitions.
- **utils.py:** Core utility module covering the NLP pipeline, model inference, severity scoring, and PDF generation.
- **requirements.txt:** All Python dependency specifications with pinned versions.
- **Dockerfile:** Docker container build instructions including system-level dependencies like Tesseract.
- **Procfile:** Gunicorn startup command used by Railway for production deployment.
- **.python-version:** Pins the Python 3.11 runtime for the deployment environment.
- **templates/medpredict_ui.html:** The frontend HTML, CSS, and JavaScript interface rendered by Flask.
- **models/** Contains all six serialized model files: best_model.pkl, decision_tree.pkl, naive_bayes.pkl, neural_network.keras, symptom_list.pkl, and label_encoder.pkl.
- **data/** Contains the three supplementary CSV files: symptom_Description.csv, symptom_precaution.csv, and Symptom-severity.csv.

12. Challenges and Solutions

12.1 Natural Language Symptom Extraction

Challenge: People describe their symptoms using all kinds of words and phrases that look nothing like the standardized symptom names in the dataset. A simple keyword search would miss most of what users actually type.

Solution: I built a three-stage pipeline that first checks a custom alias dictionary I created manually, then does direct string matching, and finally uses RapidFuzz fuzzy matching on word n-grams. The alias dictionary took the most time to build but makes the biggest difference in practice, because it handles the cases where the user's words are semantically equivalent to a symptom but look completely different on the surface.

12.2 PDF and Image Document Processing

Challenge: Uploaded documents come in many different formats and qualities. Some PDFs have native text that can be extracted directly, while others are scanned images where OCR is required. OCR quality depends a lot on the image resolution and font.

Solution: I implemented a dual-engine approach for PDF extraction - PyMuPDF runs first, and if it does not extract useful text, pdfplumber tries as a fallback. For image files, pytesseract handles OCR with PIL managing the image loading. The extracted text from any of these paths then goes through the same NLP pipeline.

12.3 Cross-Environment Model Compatibility

Challenge: The models were trained and tested locally on Windows, but the production environment runs on a Linux container. Making sure everything worked the same in both environments required careful attention to how files were saved and how library versions were specified.

Solution: I saved all scikit-learn models using joblib and the Keras model in TensorFlow's native .keras format, both of which are platform-independent. The Dockerfile was set up to install all system-level dependencies alongside the Python packages, so the container environment exactly mirrors what the models expect.

12.4 PDF Report Generation

Challenge: Generating a professional-looking PDF report from code is more complex than it sounds. Getting the layout, fonts, tables, and page breaks to look right required a lot of fine-tuning.

Solution: I used ReportLab's platypus layout engine, which gives granular control over every element of the document. I defined custom paragraph styles using a navy and blue color scheme, structured the prediction results into a clean table, and made sure the medical disclaimer was clearly visible at the bottom of the report.

13. Conclusion

MedPredict AI successfully brings together machine learning, deep learning, natural language processing, and modern web deployment into a single, fully working application that anyone can use. Every goal I set for this project was met.

All four models - Decision Tree, Random Forest, Naive Bayes, and the Neural Network - were trained, evaluated, and integrated into a live system, all achieving 100% accuracy on the test set. The NLP pipeline handles natural language symptom input in a way that feels intuitive to the user. The document upload feature lets people with existing medical records get predictions without typing anything. The web interface is clean and easy to use, and the cloud deployment means the system is available to anyone with an internet connection. The PDF export gives users something they can physically take to a doctor's appointment.

From a software engineering perspective, the project is structured cleanly with a clear separation between the API layer in `app.py` and the logic in `utils.py`. The containerized deployment ensures consistency across environments, and the RESTful API design makes it easy to extend or integrate with other systems in the future.

The 100% test accuracy is a result of the dataset's clean and structured nature rather than proof of clinical-grade performance. In a real-world setting with messier patient data, results would differ. But the architecture and pipeline built here are solid enough to serve as a foundation for a more robust system if it were extended with real clinical data and validated by medical professionals.

14. Future Work and Recommendations

There are several directions this project could go next:

- **Real-World Dataset Integration:** Training on larger, real clinical datasets with comorbidities, incomplete records, and noisy labels would make the models much more generalizable.

- **Expanded Disease Coverage:** The current 41-disease scope could be expanded significantly by adding datasets for rare diseases, chronic conditions, and mental health disorders.
- **Conversational AI Interface:** Replacing the text input with a chatbot interface powered by a large language model would make symptom collection feel even more natural and guided.
- **Lab Result Integration:** Adding basic lab values like blood counts or glucose levels as features alongside symptoms would meaningfully improve prediction quality.
- **Mobile Application:** Building a native iOS and Android app would make the system more accessible, especially in areas where mobile internet access is more common than desktop use.
- **Multilingual Support:** Adding Urdu and other regional languages to the alias dictionary and NLP pipeline would make the system much more useful in Pakistan and other multilingual markets.
- **Clinical Validation:** Working with medical professionals to test the system's outputs against real patient cases would be an essential step before any kind of medical deployment could be considered.
- **Explainability Module:** Adding SHAP or LIME visualizations to show which symptoms most influenced each prediction would make the system more transparent and trustworthy.

15. References

1. Kaggle Dataset: Disease Symptom Description Dataset.
<https://www.kaggle.com/datasets/itachi9604/disease-symptom-description-dataset>
2. Pedregosa et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
3. Abadi et al. (2016). TensorFlow: A System for Large-Scale Machine Learning. OSDI.
4. Flask: Web Development, one drop at a time. <https://flask.palletsprojects.com/>
5. RapidFuzz: Rapid fuzzy string matching in Python.
<https://github.com/rapidfuzz/RapidFuzz>

6. ReportLab PDF Library. <https://www.reportlab.com/>
7. PyMuPDF: A Python binding for MuPDF. <https://pymupdf.readthedocs.io/>
8. Smith, R. (2007). An Overview of the Tesseract OCR Engine. Ninth International Conference on Document Analysis and Recognition (ICDAR).
9. Docker: Enterprise Container Platform. <https://www.docker.com/>
10. Railway Cloud Platform. <https://railway.app/>

— *End of Report* —